



Verifying Numerical Methods with Isabelle/HOL

Dustin Bryant
Independent, USA

Jonathan Julian Huerta y Munive
Czech Technical University in Prague, Czechia

Simon Foster
University of York, UK

November 2025

Abstract

Modern machine learning pipelines are built on numerical algorithms. Reliable numerical methods are thus a prerequisite for trustworthy machine learning and cyber-physical systems. Therefore, we contribute a framework for verified numerical methods in Isabelle/HOL based on ITrees. Our user-friendly specification language enables the direct declaration of numerical programs that can be annotated with variants and invariants for reasoning about correctness specifications. The generated verification conditions can be discharged via automated proof methods and lemmas from the HOL-Analysis library. The ITrees foundation interacts with Isabelle’s code generator to export source code. This provides an end-to-end path from formal specifications with machine-checked guarantees to executable sources. We illustrate the process of modelling numerical methods and demonstrate the effectiveness of the verification by focusing on two well-known methods, the bisection method and the fixed-point iteration method. We also contribute crucial extensions to the libraries of formalised mathematics required for this objective: higher-order derivatives and Taylor’s theorem in Peano form. Finally, we qualitatively evaluate the use of the framework for verifying numerical methods.

1 Introduction

Numerical methods are an important class of algorithms that iteratively approximate the solutions to mathematical problems. They often employ



concepts from linear algebra and multivariate analysis, and are foundational to machine learning. For instance, stochastic gradient descent, the backbone of modern training of neural networks, minimizes cost functions by iteratively approximating their gradients and modifying the networks' weights accordingly. While such algorithms have strong mathematical foundations, work on their formal verification is still in its infancy. The reason for this is that their verification is a challenging task. It requires the usage of libraries of diverse formalised mathematical concepts, such as real numbers, matrices, transcendental functions, and their derivatives.

Two recent developments make this task more feasible. The advancement of formalised libraries of Analysis and Ordinary Differential Equations [10, 12, 15, 19] and the development of libraries for program verification on top of these [7, 8, 11]. Although various iterative methods have been formalised before [17, 23, 14], the existing Taylor-based proofs typically rely on the interval-based Lagrange remainder, thereby requiring stronger hypotheses than necessary and adding avoidable proof overhead. Therefore, we ponder whether verification of numerical methods with more relaxed conditions and leveraging modern verification frameworks is feasible.

To answer this, we contribute a method for verifying numerical algorithms using interactive and automated theorem proving. Our approach uses the Isabelle/ITree [8] framework to provide Hoare logic and verification condition generation (VCG) for algorithms operating over the real numbers, including total correctness. Discharging the resulting VCs requires application and development of foundational theorems from multivariate analysis, many of which are provided by Isabelle's **HOL-Analysis** package. Isabelle's **Sledgehammer** tool largely aids in the proving process, and substantially automates the verification. Moreover, the use of ITrees means we can execute the algorithms in Isabelle and generate code that intrinsically has a high degree of assurance.

We apply our framework to the verification of two numerical algorithms: the bisection method and the fixed-point method. The latter requires special versions of higher-order derivatives and Taylor's theorem with the Peano remainder. We use these algorithms to qualitatively evaluate Isabelle/ITrees as a platform for verifying numerical algorithms. Our formalisation is publicly available [4] and links to it appear throughout the text as icons. 🍌

The structure of our paper is as follows. In Section 2 we introduce Isabelle/ITrees, and its extension for numerical methods. In Section 3 we consider the bisection method and verify its total correctness using our tool. In Section 4 we introduce the fixed-point method. In Section 5 we give an overview of differentiation within Isabelle before motivating and defining a



theory of higher-order differentiation. In Section 6 we apply this obtained theory of differentiation to prove the Taylor Theorem with Peano Remainder and show how it relates to the existing Taylor theorem with Lagrange remainder. In Section 7 we apply both of these calculus developments to verify the correctness of three convergence cases of the fixed-point method. In Section 8, we summarize the experience of using VCG to verify programs in Isabelle; we highlight metrics and emphasise theorems necessary for this verification process. In Section 9, we show how this study relates to other research, and in Section 10, we draw our conclusions and discuss future directions.

2 VCG for Numerical Methods

Our verification approach is based on the Isabelle/ITrees framework [8], which is part of the Isabelle/UTP semantics and verification ecosystem [7]. The library provides an abstract imperative program notation, mechanisms for declaring partial and total correctness specifications, (in)variant annotations, and a verification condition generator. The programs are assigned a formal semantics in terms of Interaction Trees (ITrees), which unify denotational and operational semantics for programs. In particular, ITrees, as coinductively defined data structures, are fundamentally executable in nature, and the library leverages Isabelle's code generator for this. We extend the existing library to support real number variables by combining it with the HOL-Analysis package.

Consider the following imperative implementation of scalar multiplication for vectors in Isabelle/ITrees: 

```

1 alphabet 'i st =
2   i :: nat
3   vc :: "real vec['i]"
4
5 program vec_scale "(n::real, X::real vec['n])" =
6   "vc := X;
7   i := 0;
8   while i < CARD('n) do
9     vc[i] := n * X(i);
10    i := i + 1
11 od"

```

We use the **alphabet** command to declare a state space for the program, consisting of two variables: i of natural number type, and vc of vector type. The state type st is parametric over $'i$, which gives the dimension of the

vector under consideration. For ease of presentation, we omit several type annotations; however, `'i` must be a type of finite cardinality corresponding to a natural number.

We define program `vec_scale` using the `program` keyword. The program's specification is similar to Dijkstra's guarded command language. The program takes as input a real number n , which is the scale, and a vector X of dimension `'n`. Having initialised vc and i , the program iterates through each element of the vector, performing a point-wise multiplication. The notation `CARD('n)` gives the dimension of the vector as a natural number, $X(i)$ retrieves the i th element of vector X .

We can run the program using the `execute` keyword:

```
1 execute "vec_scale(3, Vector[1,2,3,4])"
```

where the `Vector` constructor builds a corresponding input to the program via a list and infers the dimension (i.e. four). Then, the `vec_scale` receives that vector and the scaling factor 3. The command first compiles the program to an ITree, generates SML code for this, and then executes the SML code. Isabelle reports that the program terminates with output state `i = 3, vc = Vector[3, 6, 9, 12]`.

We can also verify such a numerical algorithm using Hoare logic. The Hoare triple below states a total correctness specification claiming that the program terminates and in the final state `vc` is indeed the result of a scalar multiplication, which is provided by the Isabelle function `*R`:

```
1 lemma "H[True] vec_scale(n, X) [vc = n *R X]"
```

We can verify such a specification using a proof method called `vcg`, which applies the laws of Hoare logic to produce verification conditions to the program. This usually also requires that we annotate the program with loop variants and invariants. In this case, a sufficient invariant for the loop is $i \leq \text{CARD}('n) \wedge (\forall k < i. vc(k) = n * X(k))$, which we add as an annotation to the loop: 

```
1 while i < CARD('n)
2 invariant i ≤ CARD('n) ∧ (∀ k < i. vc k = n * X k)
3 variant CARD('n) - i do ... od
```

With this, the `vcg` method yields a single proof obligation

$$\forall k < \text{CARD}('n). vc(k) = n * X(k) \implies vc = n *_{\text{R}} X$$

which is the extensionality principle for vectors. If each element of the vector



vc is n multiplied by the corresponding element of X , then vc is the scalar multiplication of n by X . This VC can be discharged automatically by application of the Sledgehammer tool.

3 Formalising the Bisection Method

Here, we move to providing evidence that the framework is suitable for the verification of numerical algorithms. The bisection method, in particular, is a classic, derivative-free algorithm for finding roots of continuous functions. It is a common entry point to the study of numerical methods for many mathematicians due to its simplicity. We therefore take the bisection method as our first nontrivial program, proving its correctness within the VCG framework. Below, we first describe its implementation, then justify the correctness specification and its loop invariants, and finally discuss its proof, all while highlighting features of our framework.

3.1 The Algorithm

Recall that Bolzano's Theorem states that a real-valued function f , continuous on a closed interval $[a, b] \subseteq \mathbb{R}$, and such that $f(a)$ and $f(b)$ have opposite signs, has a root inside the open interval (a, b) . The bisection method merely iterates this aforementioned fact (see Listing 1). That is, at each step n , it defines the current bracket as the interval $[l_n, u_n]$ that halves the previous bracket (where $[l_0, u_0] = [a, b]$). The bisection method evaluates f at the bracket's midpoint $m_n = (l_n + u_n)/2$, and retains the bracket's half on which the sign change persists. It repeats this process until the bracket's length falls below a prescribed threshold. The algorithm is formalised below using Isabelle/ITrees: 

```

1 program bisection "(f :: real ⇒ real, a :: real, b :: real, tol
  :: real)" over state
2 = "iter := 0;
3   fa := f(a);
4   fb := f(b);
5   lower := a;
6   upper := b;
7   xmid := lower;
8   ymid := f(xmid);
9   while upper - lower > tol
10  do
11    iter := iter + 1;
12    xmid := (lower + upper)/2;
13    ymid := f(xmid);

```

```

14   if fa*ymid > 0
15   then lower := xmid; fa := ymid
16   else upper := xmid; fb := ymid
17   fi
18 od"

```

Listing 1: Bisection Method in Isabelle

As an example, we can use the method to calculate an approximate value for $\sqrt{2}$. We do this by setting $f(x) = x^2 - 2$, and setting the initial bracket to $[1, 1.5]$, which establishes an initial guess for the root since $f(1) < 0 < f(1.5)$. We can then execute the algorithm with a tolerance of 0.0001, as show below:

```

1 execute "bisection ( $\lambda$  x. x^2 - 2, 1, 1.5, 0.0001)"

```

The command generates code, executes the Bisection method, and reports the approximate root of 1.41421508789, which is calculated after 13 iterations.

We now proceed to describe the method in more detail. First, we set an initial bracket $[\text{lower}, \text{upper}] = [a, b]$ and we use **fa** and **fb** to represent the function value at the left and right limits of our current bracket. The **while** loop checks the condition that the bracket's size is strictly larger than the input tolerance (**upper** - **lower** > **tol**). Inside the loop, we calculate the midpoint of our current bracket, **xmid** := (**lower** + **upper**) / 2, and the value of the input function f at that midpoint, **ymid**. If **ymid** has the same sign as the bracket's left-end value — **if** **fa*****ymid** > 0 — then the algorithm redefines the left end of the bracket and its value so that the bracket shrinks by half while maintaining opposite sign values on the bracket ends — **then** **lower** := **xmid**; **fa** := **ymid**. Otherwise, if **ymid** does not have the same sign as **fa**, then it must be that **fb** * **ymid** $\geq$ 0 so that either **fb** and **ymid** have the same sign or **fb** is a root. Either way, the algorithm replaces the right end of the bracket and its value — **else** **upper** := **xmid**; **fb** := **ymid**. These conditional actions preserve the assumptions that enable the continuous application of Bolzano's Theorem. Thus, at each iteration, there is at least one root within the bracket by Bolzano's Theorem or because an endpoint is itself a root.

3.2 Total Correctness and Loop Invariants

With the intuition for the correctness of the algorithm established, we state the principal result below. It asserts the method's correctness and guarantees its termination, while also stating the theorem's underlying assumptions. 🍷

```

1 theorem bisection_error_bound:
2   assumes positiv_tolerance: "tol > 0"
3   assumes sufficiently_small_tol: "tol < b - a"
4   assumes continuous_f: "continuous_on {a..b} f"
5   assumes opposite_signs: "f(a) * f(b) < 0"
6   shows "H[True] bisection(f,a,b,tol)
7     [∃ c::real. f(c) = 0
8       ∧ a < c ∧ c < b
9       ∧ upper - lower ≤ tol
10      ∧ |c - xmid| ≤ (b - a) / 2^iter
11      ∧ |c - xmid| ≤ tol
12      ∧ iter = ⌈log 2 ((b - a) / tol)⌉]"

```

Listing 2: Total correctness of Bisection Method

The theorem in Listing 2 asserts that our program terminates with the value of `xmid` being an approximate root of f . That is, it is at most a distance of `tol` to the actual root c (i.e. $|c - \text{xmid}| \leq \text{tol}$) after precisely $\lceil \log_2 \left(\frac{b-a}{\text{tol}} \right) \rceil$ iterations, provided f is continuous on $[a, b]$. We assume continuity on $[a, b]$ and $f(a) \cdot f(b) < 0$ so that we may apply Bolzano's theorem. We choose the assumptions `tol > 0` and `b - a > tol` as minimal conditions guaranteeing a nonempty initial domain and the possibility of termination.

We formally state the program's loop invariants below. 🍷

```

1 while upper - lower > tol
2 invariant fa = f(lower) ∧ fb = f(upper)
3   ∧ (lower = xmid ∨ upper = xmid)
4   ∧ a ≤ lower ∧ upper ≤ b ∧ lower < upper
5   ∧ fa * fb ≤ 0
6   ∧ upper - lower = (b - a) / 2^iter
7   ∧ (iter = 0 ∨ 2 * (upper - lower) > tol)
8   variant nat(⌈log 2 ((b - a) / tol)⌉) - iter
9 do ... od

```

The conjuncts `fa = f(lower) ∧ fb = f(upper)` assert that `fa` (respectively `fb`) is always the evaluation of the input function at the lower (respectively upper) portion of the bracket. This corresponds to the implementation's lines

```

1 fa := f(a);
2 fb := f(b);
3 ...
4 then lower := xmid; fa := ymid
5 else upper := xmid; fb := ymid

```

regardless of which portion of the bracket was most recently replaced. We

capture this replacement in the invariant with the disjuncts $\text{lower} = \text{xmid} \vee \text{upper} = \text{xmid}$. That is, either the **then** branch of the **if-then-else-fi** command is traversed or the **else** branch is. The interval $[\text{lower}, \text{upper}]$ shrinks under the loop while satisfying the conjunction $\text{a} \leq \text{lower} \wedge \text{upper} \leq \text{b} \wedge \text{lower} < \text{upper}$, which guarantees that it is always a nonempty subinterval of $[a, b]$. The inequality $\text{fa} * \text{fb} \leq \mathbf{0}$ ensures that each new bracket is chosen in a manner such that Bolzano’s Theorem is still applicable. Finally, we account for the rate of decay of the interval with the invariant $\text{upper} - \text{lower} = (b - a)/2^{\text{iter}}$, which says that at each iteration the bracket is halved and is the basis for the name *bisection* method.

From the invariant’s first seven conjuncts alone, we can show most of the postcondition in Listing 2. Yet, we can only show a relaxed version of the postcondition’s last conjunct, namely

$$\text{iter} \geq \left\lceil \log_2 \left(\frac{\text{b} - \text{a}}{\text{tol}} \right) \right\rceil.$$

We can only show that the method takes at least that many iterations to ensure that the root and the bracket’s midpoint approximation lie in the interval $[\text{lower}, \text{upper}]$ with at most tol width. Yet, we know that the algorithm should terminate when $\neg(\text{upper} - \text{lower} > \text{tol})$. Thus, we expect iter to not be larger than necessary and the above inequality to be, in fact, an equality.

To obtain an invariant that aides in proving the equality, we first observe that during the penultimate step, the **while**-condition still holds; that is,

$$\text{upper}_{\text{iter}-1} - \text{lower}_{\text{iter}-1} > \text{tol}.$$

As the bracket halves at each step, for $\text{iter} > 0$, we have

$$\text{upper}_{\text{iter}-1} - \text{lower}_{\text{iter}-1} = 2 \cdot (\text{upper}_{\text{iter}} - \text{lower}_{\text{iter}}).$$

Since $\text{iter} - 1$ is undefined when $\text{iter} = 0$, the expression

$$\text{iter} = 0 \vee 2(\text{upper}_{\text{iter}} - \text{lower}_{\text{iter}}) > \text{tol}$$

is almost the desired invariant, but since the program variables do not record their history, the invariant is:

$$\text{iter} = 0 \vee 2(\text{upper} - \text{lower}) > \text{tol}.$$

That is, “either the previous interval satisfied the **while**-condition *or* the algorithm is at its first step.” Since the **while**-guard holds at the penultimate

step but not the last, then

$$\frac{b-a}{2^{\text{iter}}} \leq \text{tol} < \frac{b-a}{2^{\text{iter}-1}}.$$

It follows that **iter** is the least k with $(b-a)/2^k \leq \text{tol}$, and hence $\text{iter} = \lceil \log_2((b-a)/\text{tol}) \rceil$.

3.3 Verifying the Bisection Method

With the aforementioned loop invariants, our proof method **vcg** generates a total of 22 goals, 20 of which can be proven directly with Isabelle's **Sledgehammer** tool. We list one of those goals below as a representative example.

```
1 fix {iter lower upper}
2 assume a0: "upper - lower = (b - a) / 2 ^ iter"
3 assume a1: "tol < (b - a) / 2 ^ iter"
4 show "tol < 2 * upper - (2 * lower + 2 * upper)/2"
5 by (smt (z3) a0 a1 field_sum_of_halves)
```

Sledgehammer easily discharges the proof obligations for this goal using already proven facts from Isabelle's libraries. Another 19 goals are similarly discharged. The remaining two goals are the same up to symmetry, we state one here:

```
1 fix iter lower upper
2 assume a0: "¬ (tol < (b - a) / 2 ^ iter)"
3 assume a1: "f lower * f upper ≤ 0"
4 assume a2: "a ≤ lower"
5 assume a3: "upper ≤ b"
6 assume a4: "upper - lower = (b - a) / 2 ^ iter"
7 assume a5: "tol < 2 * upper - 2 * lower"
8 show "∃c. f c = 0 ∧ a < c ∧ c < b ∧ |c - upper| ≤ (b - a) / 2 ^
    iter ∧ |c - upper| ≤ tol ∧ int iter = ⌈log 2 ((b - a) / tol)⌈"
```

This goal captures the intuition behind the final invariant. Namely, at the penultimate iteration, the bracket is wider than the tolerance, $\text{tol} < 2*\text{upper} - 2*\text{lower}$, but after another iteration, the bracket will be within error tolerance, $\neg (\text{tol} < (b-a)/2^{\text{iter}})$. Since the signs of f on the final bracket's boundaries are proper, $f \text{ lower} * f \text{ upper} \leq 0$, there is a root, by Bolzano's Theorem, inside the bracket. With algebraic manipulations, the value for the number of iterations is established.

In this section, we have seen that our framework enables the straightforward formalisation of the bisection method and its correctness specification and loop invariants. Furthermore, the framework's **vcg** proof method reduces



the bisection method's proof of correctness to a *single*, up to symmetry, non-trivial goal proven by application of Bolzano's Theorem and some algebra. Thus, the verification process, even for this nontrivial program, is highly automated.

4 Formalising the Fixed-Point Method

We now focus on another important, nontrivial numerical method, the fixed-point method (FPM). Similar to the previous section, we discuss the algorithm's formalisation, its correctness specification, and the challenges and intuitions in proving it. The FPM requires us to contribute novel formalisations to the libraries of formalised mathematics in Isabelle. In particular, an implementation of higher-order differentiability (Section 5) and Taylor's theorem (Section 6).

As the name suggests, the FPM approximates *fixed points* of its input functions. These are points r such that $f(r) = r$. It can also be seen as a root finding method since $f(r) = r$ is equivalent to $f(r) - r = 0$. The fixed-point method starts from an initial guess x_0 and repeatedly applies f : $x_{n+1} = f(x_n)$. However, it relies on the assumption that the input function f reduces the distance between points in a neighbourhood S of x_0 after each application. As before, the program's formalisation is straightforward in the framework: 

```

1 program fixed_point_iter "(
2   f :: real ⇒ real,
3   x0 :: real,
4   tol :: real,
5   max_iter :: nat
6 )" over state
7 = "x := x0; x_new := f x; itr := 0; Break := 0;
8 while (itr < max_iter ∧ Break = 0)
9 do
10   if |x_new - x| < tol then Break := 1 fi;
11   x := x_new; x_new := f x; itr := itr + 1
12 od"

```

Listing 3: Fixed-Point Method in Isabelle

In Listing 3, the variable $\mathbf{x}$ represents the previous value of the approximation x_n , while $\mathbf{x_new}$ corresponds to the most recent one, $x_{n+1} = f(x_n)$. The variable $\mathbf{itr}$ counts the number of iterations. Each iteration checks whether the distance between the previous and the new approximations of the fixed point is less than the input tolerance. The distance test $|\mathbf{x_new} -$



$x_i < \text{tol}$ serves for indicating whether the loop should stop at the start of the next iteration, which is configured by the assignment $\text{Break} := 1$. Regardless of this test's outcome, the body of the loop updates the program's variables ($x := x_new$, $x_new := f\ x$, and $\text{itr} := \text{itr} + 1$). The loop ends if the latest distance is less than the tolerance or if a maximum number of iterations has been reached.

Consider the following example invocation, which computes an approximate value for $\sqrt{3}$:

```
1 execute"fixed_point_iter( $\lambda x.(3/x+x)/2,1,0.001,10$ )"
```

This terminates in only four iterations, with $x = 1.73205081001$. Moreover, as we shall demonstrate later, this convergence is quadratic. Computing $\sqrt{3}$ amounts to root-finding for the equation $f(x) = x^2 - 3$, for which we apply Newton's Method, a special case of the Fixed-Point Method with quadratic convergence. This yields the fixed-point iteration

$$x_{k+1} = g(x_k), \quad g(x) = x - \frac{f(x)}{f'(x)} = \frac{1}{2} \left(x + \frac{3}{x} \right).$$

In this essay, we do not verify properties specialised to Newton's method but we note that the g above satisfies conditions which we verify in a later section for quadratic convergence with the fixed-point method. Thus, if one accepts that Newton's Method is a FPM, the claims follow and agree with the example illustrated above. While the bisection method is guaranteed to converge, as we shall show, the FPM converges more rapidly under certain conditions. However, for the FPM to converge at all, it must be applied within a contractive neighbourhood. Formally, if there is a constant c such that

$$|g(x) - g(y)| \leq c |x - y| \text{ and } 0 < c < 1,$$

for $x, y \in S$, then the function g is c -contractive on S , or equivalently, S is a contractive set for g . If x_0 is chosen inside a c -contractive neighbourhood of r , then each x_n will remain inside it. This is formalised in the following result:

```
1 lemma contraction_ball_closure:
2   fixes f :: "real  $\Rightarrow$  real" and r c  $\delta$  :: real
3   assumes fr: "f r = r"
4     and contr: " $\forall s\ t. |s - r| < \delta \longrightarrow |t - r| < \delta \longrightarrow |f\ s - f\ t|$ 
5        $\leq c * |s - t|$ "
6     and c_bound: " $0 \leq c \wedge c < 1$ "
7   shows " $\forall n\ x. |x - r| < \delta \longrightarrow |(f^{^^n})\ x - r| < \delta$ "
```

Intuitively, the fact that each x_n remains in a contractive neighbourhood is half of what is needed to show that the FPM converges. The other half requires observing that

$$|x_n - r| = |f(x_{n-1}) - f(r)| \leq c |x_{n-1} - r|,$$

and, by inductively applying the contraction law, we get that x_n converges to r since $|x_n - r| \leq c^n |x_0 - r|$, for all $n \geq 0$, and c^n converges to 0.

Therefore, the general strategy for proving results about the FPM consists of establishing conditions under which a contractive neighbourhood exists. In our proof, these conditions are related to the higher-order differentiability of f and the possibility of approximating f via a Taylor expansion. In the spirit of weakening the assumptions for the proof of correctness of the FPM, we focus on generalising these concepts in the upcoming sections. Specifically, previous formalisations of Taylor’s theorem in Isabelle are for its Lagrange form, where our argument uses the Peano form. As we shall show, since the conditions for the Lagrange form imply those of the Peano form, our argument ends up being more general than it would have been by reusing previous formalisations. Thus, we take a detour into our contributions to the libraries of mathematical analysis.

5 Higher-Order Differentiation

In this section, we present a precise notion of higher-order differentiability. The most common approach for formalising differentiation in modern proof assistants is to introduce a relation R between functions such that $R(f, g, F)$ holds if g is the derivative of f “according” to the filter F , where filters are generalisations of sequences to more abstract spaces. Since the derivative is unique for f , it is also customary in Isabelle to introduce a term Df using Hilbert’s choice operator (ε) , i.e. $Df \triangleq (\varepsilon g. R(f, g, F))$. Examples of these constructions from the HOL-Analysis library are the R — D pairs:

- `has_derivative` — `frechet_derivative`,
- `has_vector_derivative` — `vector_derivative`,
- `has_field_derivative` — `deriv`.

For example, we can prove the following facts certifying the derivative of $f(x) = x^2$:

```
1 ((λx. x^2) has_field_derivative (2 * y)) (at y)
2 deriv (λx. x^2) = (λy. 2 * y)
```

The first property states that, at any point $y \in \mathbb{R}$, the derivative of f is $2 \cdot y$. The second property allows us to calculate the derivative function as a consequence of the first one.

A third predicate modelling *differentiability* is also introduced, and lemmas relating these notions are proven. For instance, the HOL-Analysis theorem **frechet_derivative_works** states the equivalence

$$f \text{ differentiable } F \longleftrightarrow (f \text{ has_derivative } D f) F,$$

where we abbreviate $D f \triangleq \text{frechet_derivative } f F$. For a general formalisation of higher-order differentiability, we follow this approach while reusing previous library constructs. For presentation purposes, we focus here on our results specialised to the real numbers, although our formalised definitions are general and hold for normed vector spaces over the reals.

First, we reuse the **deriv** operator to recursively define the corresponding higher-order operator:

```
1 fun Nth_derivative :: "nat  $\Rightarrow$  (real  $\Rightarrow$  real)
2    $\Rightarrow$  (real  $\Rightarrow$  real)" where
3   "Nth_derivative 0 f = f " |
4   "Nth_derivative (Suc n) f
5   = deriv (Nth_derivative n f)"
```

Listing 4: Higher-Order derivative

This definition is consistent with previous implementations: we show that it coincides with **deriv** iterated n times $((\text{deriv} \text{ ^^ } n) f)$.

Next, in contrast to previous implementations of *higher-order differentiability* in Isabelle’s Archive of Formal Proofs (AFP) [5, 16], our definition (Listing 5) assumes minimal requirements. As an example, consider the most common, limit-based characterization of the (second) derivative at a point:

$$f''(x) = \lim_{h \rightarrow 0} \frac{f'(x+h) - f'(x)}{h}.$$

The presence of the term $f'(x+h)$ implicitly entails the existence of f' in a neighbourhood of x . However, it does not imply the continuity of f' , nor its differentiability on the same domain where f is. It only requires differentiability of f' at x . With that in mind, we recursively define the notion that f is *k-times differentiable at a*: a function f is always zero times differentiable at a with $f^{(0)} = f$, and it is $(k+1)$ -times differentiable at a provided f is k -times differentiable at every point in a neighbourhood of a and $f^{(k+1)}(a) = (D f^{(k)})(a)$. We showcase the formalisation of this below. 🎉

```

1 primrec k_times_differentiable_at :: "nat
2   ⇒ (real ⇒ real) ⇒ real ⇒ bool"
3   where "k_times_differentiable_at 0 f a ⟷ True"
4   | "k_times_differentiable_at (Suc k) f a ⟷
5     (∃ε>0. (∀x. |x - a| < ε
6       → k_times_differentiable_at k f x))
7     ∧ (Nth_derivative k f has_derivative
8       (λh. Nth_derivative (Suc k) f a * h)) (at a)"

```

Listing 5: Higher Differentiable Defined

Together, the predicate `k_times_differentiable_at` and the operator `Nth_derivative` provide a *local* specification for higher differentiability at a point and a *canonical witness* for higher derivatives. We prove several results relating them to each other and to concepts from previous libraries. Then, we use our definitions as the basis for further constructs. For instance, we define the higher-order differentiability of f on a set S as expected: 🧠

$$\begin{aligned}
 & \text{k_times_differentiable_on } k \ f \ S \\
 & \iff (\forall x \in S. \text{k_times_differentiable_at } k \ f \ x).
 \end{aligned}$$

We then show that this definition, on the reals, is more general than that in the Smooth Manifolds AFP entry [16]. 🧠

```

1 lemma high_diff_on_imp_k_times_on:
2   fixes f :: "real ⇒ real"
3   assumes "open S"
4   shows "higher_differentiable_on S f (Suc n)
5     ⇒ k_times_differentiable_on (Suc n) f S"

```

That is, while the Smooth Manifold’s definition implies ours, the converse does not hold since, for first-order differentiability, our definition would require continuity of the first derivative. A similar generalisation result for more abstract spaces than the real numbers is not in the scope of this work. Yet, we have shown that the definition of continuous differentiability C^k on a set U from the AFP [5] coincides with the Smooth Manifold’s higher-order differentiability definition, provided that U is open: 🧠

```

1 lemma higher_differentiable_on_real_iff_Ck_on:
2   fixes f :: "real ⇒ real" and U :: "real set"
3   assumes Uop: "open U"
4   shows "higher_differentiable_on U f k ⟷ C_k_on k f U"

```

This illustrates the intention behind that entry’s definition of higher differentiability. Additionally, we have related our definition with the `C_k_on` definition [5]. Specifically, we have shown that if a function is $(n+2)$ -differentiable

at a point x , then it is n -times continuously differentiable (C^n) at a neighbourhood of x : 

```

1 lemma SucSucn_times_diff_imp_Cn_on:
2   fixes f :: "real ⇒ real" and n :: nat
3   assumes diff_at:
4     "k_times_differentiable_at (Suc (Suc n)) f x"
5   shows "∃ε>0. C_k_on n f {x - ε <..< x + ε}"

```

Besides proving the relative position of our definitions compared to previous works, we have also focused on providing library lemmas and simple proof automation for them. In particular, we have proved linearity and compositionality of our various definitions:

1. For all k such that $1 \leq k \leq n$,

$$(f^{(n-k)})^{(k)}(x) = (f^{(k)})^{(n-k)}(x) = f^{(n)}(x),$$

2. For all $x, c \in \mathbb{R}$,

$$(c \cdot f)^{(n)}(x) = c \cdot f^{(n)}(x).$$

3. For all $x \in \mathbb{R}$,

$$(f + g)^{(n)}(x) = f^{(n)}(x) + g^{(n)}(x).$$

We have also proved Leibniz's formula for the derivative of a product of functions. That is, if f and g are n -times differentiable at x , then their product is as well, and the n^{th} -derivative of $f \cdot g$ is given by:

$$(f \cdot g)^{(n)} = \sum_{k=0}^n \binom{n}{k} f^{(k)}(x) g^{(n-k)}(x).$$

To formalise these results we have consulted online lecture notes and classical textbooks in calculus [18, 27].

We use a well-known method in Isabelle for configuring proof automation of derivative operators [11]. That is, we strategically collect standard facts about the higher-order derivative operator into two rule sets. The first, **kdiff**, provides *closure/transfer* rules for our (higher-order) differentiability predicate while the second, **kderivs**, records *equational* laws for our derivative operator. Using these two collections, Isabelle's simplifier automates routine algebraic certification of higher derivatives. This implies that even nontrivial expressions simplify almost immediately once unfolded. The deliberately complicated example below demonstrates this, where **Sledgehammer**

can certify the value of a fifth derivative of a composite polynomial-sum expression only after supplying the automation laws to the simplifier.

```

1 lemma demo_all_kderivs:
2   fixes x :: real
3   defines "F i ≡ (λt::real. ((-1) ^ i) * (((of_nat (i + 1)) * t) ^
4     i))"
5   defines "P ≡ (λt::real. (3 * t - 5) * (t ^ 2))"
6   defines "Q ≡ (λt::real. - (2 * t) + 7)"
7   defines "H ≡ (λt::real. P t + (Σ i≤0. F i t) - Q t + (t ^ 4 - 3
8     * (t ^ 4)))"
9   shows "Nth_derivative 5 H x = 0"
10  unfolding assms
11  by (simp add: kdiff kderivs, smt (verit, best) One_nat_def Suc_1
    add commute add.left_commute
    add_diff_cancel_right' mult_eq_0_iff numeral_Bit1
    one_plus_numeral plus_1_eq_Suc
    sum.neutral zero_neq_numeral)

```

Listing 6: Example with automated derivative certification

Together, the contributions from this section span approximately 2160 lines of Isabelle code (without counting white spaces or comments).

6 Taylor’s Theorem in Peano Form

We go back to the main reason for our development of higher-order differentiation and the focus of this section: motivating and proving Taylor’s theorem with the Peano remainder. Notably, Isabelle already has a formalisation of a version of Taylor’s theorem in its theory file **MacLaurin.thy**:

```

1 theorem Taylor:
2   fixes a :: real and n :: nat
3   assumes INIT: "n > 0" "diff 0 = f"
4     and DERIV: "∀m t. m < n ∧ a ≤ t ∧ t ≤ b ⟶ DERIV (diff m) t :
5       > diff (Suc m) t"
6     and INTERV: "a ≤ c " "c ≤ b" "a ≤ x" "x ≤ b" "x ≠ c"
7   shows "∃t.
8     (if x < c then x < t ∧ t < c else c < t ∧ t < x)
9     ∧ f x = (Σ m<n. (diff m c / fact m) * (x - c)^m) + (diff n t /
10       fact n) * (x - c)^n"

```

Listing 7: Taylor’s Theorem with Lagrange remainder

Here, the notation **DERIV f t :> c** represents the Isabelle predicate **has_field_derivative** on *f* and *c*, with filter “**at t**”. This version is com-

monly referred to as Taylor's Theorem with the Lagrange remainder [24]. Its assumptions require that the function f and its derivatives $f^{(k)}$ (with $k \leq n$) exist on the entire interval $[a, b]$. Strictly speaking, $f^{(n)}$ only needs to exist on the open interval (a, b) with $f^{(n-1)}$ continuous on the closed interval $[a, b]$ [25]. Thus, the formalisation in Listing 7 assumes slightly more than required by demanding f to be n -times differentiable at the endpoints and by concerning itself with derivatives of order less than $n - 1$. We further note that the above lemma, **Taylor**, uses the variable **diff** as a placeholder foreseeing that its users would instantiate the derivatives for their purposes. This enables us to succinctly state and prove Taylor's theorem with Lagrange remainder using our higher differentiability predicate: 

```

1 theorem Taylor:
2   "∀t. a ≤ t → t ≤ b →
3   k_times_differentiable_at n f t
4   ⇒ [[0 < n; a ≤ c; c ≤ b; a ≤ x; x ≤ b; x ≠ c]]
5   ⇒ ∃t.
6   (if x < c then x < t ∧ t < c else c < t ∧ t < x)
7   ∧ f x =
8     (Σm<n. ((deriv^m) f c / fact m) * (x - c)^m)
9     + ((deriv^n) f t / fact n) * (x - c)^n"
10 by(rule MacLaurin.Taylor[where a=a and b=b];
11    simp; metis DERIV_deriv_iff_real_diff
12    Nth_deriv_eq_compow_deriv n_times_diff_imp_lower_deriv_diff)

```

This form is especially useful if one assumes that a function is n -times differentiable on an entire interval $[a, b]$ because it produces a convenient remainder: $R \triangleq \frac{f^{(n)}(c)}{n!}(x - c)^n$. However, when proving quadratic convergence of the FPM, we prefer to assume twice-differentiability for the input function at the fixed point, rather than on a whole interval containing r . That is, we prefer a Taylor theorem with minimal assumptions about the input function.

Let $S_n(x) \triangleq \sum_{m=0}^n \frac{f^{(m)}(c)}{m!}(x - c)^m$, and observe that, under Taylor-Lagrange's hypotheses, the remainder tends to zero as x tends to c . Indeed:

$$\begin{aligned}
f(x) &= S_{n-1}(x) + \frac{f^{(n)}(t)}{n!}(x - c)^n \quad (\text{by Taylor-Lagrange}) \\
&= \left(S_{n-1}(x) + R \right) + \left(\frac{f^{(n)}(t)}{n!}(x - c)^n - R \right) \\
&= S_n(x) + \frac{f^{(n)}(t) - f^{(n)}(c)}{n!}(x - c)^n.
\end{aligned}$$

Hence, for $x \neq c$, dividing by $(x - c)^n$ yields

$$\frac{f(x) - S_n(x)}{(x - c)^n} = \frac{f^{(n)}(t) - f^{(n)}(c)}{n!}.$$

The left-hand side of the equation above is the so-called Peano remainder, and if we further assume that $f^{(n)}$ is continuous at c , then the Peano remainder converges to 0 as x tends to c . We have formalised this as follows.

```

1 corollary Taylor_as_limit:
2   fixes f :: "real  $\Rightarrow$  real"
3   and a b c :: real and n :: nat
4   assumes npos: "0 < n"
5         and cAB: "a  $\leq$  c" "c  $\leq$  b"
6         and diff: " $\bigwedge t. a \leq t \Rightarrow t \leq b$ 
7              $\Rightarrow$  f n-times-differentiable_at t"
8         and cont: "isCont ((deriv ^^ n) f) c"
9   shows "(( $\lambda x.$ 
10 (f x -
11 ( $\sum_{m \leq n}. ((\text{deriv} ^^ m) f) c / \text{fact } m * (x - c)^m$ ))
12 / (x - c) ^ n)  $\longrightarrow$  0) (at c within {a..b})"
```

Perhaps surprisingly, this result is still derivable while:

1. forgoing that $f^{(n)}$ is continuous at c , and
2. requiring f to be n -times differentiable at c alone rather than in some neighbourhood $[a, b]$ containing c .

This is precisely what our formalisation of Taylor's Theorem with the Peano Remainder states: 

```

1 theorem Taylor_Peano_remainder:
2   fixes f :: "real  $\Rightarrow$  real" and c :: real
3   assumes diff_n : "f (Suc n)-times-differentiable_at c"
4   shows "(( $\lambda x.$  (f x - ( $\sum_{m \leq \text{Suc } n}.$ 
5     Nth_derivative m f c / fact m * (x - c)^m))
6   / (x - c)^Suc n) --c--> 0)"
```

Listing 8: Peano remainder limit

Here, the reassignment $n \mapsto n + 1$ allows us to drop the assumption $n > 0$, and the notation $f \text{ --c--> } l$ represents convergence of f to l while tending towards c .

We now provide a proof sketch of the above convergence inspired by Vitalii Konarovskiy's lecture notes [18] (Lecture 13 – L'Hospital's Rule and Taylor's Theorem – Theorem 13.4).

Proof. If one defines

$$R_n(x) \triangleq f(x) - \sum_{k=0}^n \frac{f^{(k)}(c)}{k!} (x-c)^k,$$

one can show that

$$R_n(c) = R'_n(c) = R''_n(c) = \cdots = R_n^{(n)}(c) = 0.$$

Assuming $x > c$ and repeatedly applying the Mean Value Theorem, one obtains a sequence c_j of constants such that

$$\begin{aligned} \left| \frac{R_n(x)}{(x-c)^n} \right| &= \left| \frac{R_n(x) - R_n(c)}{(x-c)^n} \right| = \left| \frac{R'_n(c_1)(x-c)}{(x-c)^n} \right| \\ &= \left| \frac{R'_n(c_1) - R'_n(c)}{(x-c)^{n-1}} \right| = \left| \frac{R''_n(c_2)(c_1-c)}{(x-c)^{n-1}} \right| \\ &\leq \left| \frac{R''_n(c_2)(x-c)}{(x-c)^{n-1}} \right| \text{ (because } c < c_1 < x) \\ &= \left| \frac{R''_n(c_2)}{(x-c)^{n-2}} \right| \\ &= \left| \frac{R''_n(c_2) - R''_n(c)}{(x-c)^{n-2}} \right| \\ &= \left| \frac{R_n^{(3)}(c_3)(c_2-c)}{(x-c)^{n-2}} \right| \leq \cdots \\ &\leq \left| \frac{R_n^{(n-1)}(c_{n-1}) - R_n^{(n-1)}(c)}{x-c} \right|, \end{aligned}$$

and $c < c_{n-1} < c_{n-2} < \cdots < c_2 < c_1 < x$.

We know that:

$$\lim_{x \rightarrow c} \frac{|R_n^{(n-1)}(x) - R_n^{(n-1)}(c)|}{|x-c|} = |R_n^{(n)}(c)| = 0.$$

We want to show that

$$\lim_{x \rightarrow c^+} \left| \frac{R_n(x)}{(x-c)^n} \right| = 0.$$

Let $\varepsilon > 0$, then there exists $\delta > 0$ such that for all $x \in (c-\delta, c)$, it is the case that

$$\left| \frac{|R_n^{(n-1)}(x) - R_n^{(n-1)}(c)|}{|x-c|} - 0 \right| < \varepsilon. \quad \dagger$$

Therefore, let us fix $x \in (c - \delta, c)$ so that the above holds.

By iterating the Mean Value Theorem on $(x, c) \subset (c - \delta, c)$, we obtain $\{c_j\}_{j=1}^{n-1} \subset (x, c)$ such that $x < c_{n-1} < \dots < c_1 < c$, and the above chain of inequalities hold. Moreover, since $(x, c) \subset (c - \delta, c)$ it follows that $\{c_j\}_{j=1}^{n-1} \subset (c - \delta, c)$, and

$$\begin{aligned} \left| \left| \frac{R_n(x)}{(x-c)^n} \right| - 0 \right| &\leq \dots \leq \frac{|R_n^{(n-1)}(c_{n-1}) - R_n^{(n-1)}(c)|}{|x-c|} \\ &< \left| \frac{|R_n^{(n-1)}(c_{n-1}) - R_n^{(n-1)}(c)|}{|c_{n-1}-c|} \right| \\ &< \varepsilon, \end{aligned}$$

where the final inequality follows from †. Therefore

$$\left| \frac{R_n(x)}{(x-c)^n} \right| \text{ converges to } 0$$

from the right as x approaches c from the right. Thus, we conclude $h_n(x) := \frac{R_n(x)}{(x-c)^n}$ satisfies $\lim_{x \rightarrow c^+} h_n(x) = 0$. By symmetry, we can prove the case when $x < c$. ■

It is worth noting that, in the proof sketch above, the particular choices of c_j depend on c , x , δ and ε ; but the final result of this theorem effectively “forgets” these data. We conclude this section by stating the desired Taylor theorem with Peano remainder: 🧠

```
1 corollary Taylor_Peano:
2   fixes f :: "real ⇒ real"
3   and a :: real and n :: nat
4   assumes "k_times_differentiable_at (Suc n) f a"
5   obtains h :: "real ⇒ real"
6   where "((λx. h x) → 0) (at a)"
7   and "f x = (Σ i ≤ Suc n.
8     Nth_derivative i f a / fact i * (x - a) ^ i
9     ) + h x * (x - a) ^ Suc n"
```

Listing 9: Taylor’s theorem with Peano remainder

Overall, Taylor’s theorem in Peano form and the results presented in this section required approximately 800 lines of Isabelle code (without counting spaces or comments).

7 Verifying the Fixed-Point Method

Having developed a theory of higher-order differentiation and formalised Taylor's theorem with Peano remainder, we proceed to derive the correctness results about the FPM. We closely follow the presentation given by Solomon [26].

As we previously discussed, if x_0 and r are elements of a c -contractive neighbourhood then x_n will be trapped in it and converge to r . We now formally state this result: 

```

1 lemma fixed_point_known_iter_error_bound_local:
2   fixes f           :: "real ⇒ real"
3   and r x0 c δ :: real
4   and max_iter :: nat
5   assumes c_nonneg: "0 ≤ c"
6   and c_strict:    "c < 1"
7   and δ_pos:      "δ > 0"
8   and r_is_fixed: "f r = r"
9   and x0_in_ball: "|r - x0| < δ"
10  and contractive: "∀s t. |s - r| < δ
11    ∧ |t - r| < δ ⟶ |f s - f t| ≤ c * |s - t|"
12  shows
13    "H[True] fixed_point_iter (f, x0, tol, max_iter) [|x - r| ≤ c^
    itr * |x0 - r| ∧ (itr ≤ max_iter)]"

```

Listing 10: Total correctness of Fixed-Point Method in a Contractive neighbourhood

The assumptions in Listing 10 define a contractive neighbourhood containing a fixed point $r = f(r)$ with x_0 sufficiently close to it. The assumption $\delta > 0$ merely stipulates that the neighbourhood properly exists.

Below, we write the loop (in)variants for the FPM:

```

1 invariant x = (f ^^ itr) x0
2   ∧ x_new = f x
3   ∧ (Break ≠ 0 ⟶ |x_new - x| < tol)
4   ∧ itr ≤ max_iter
5   variant max_iter - itr

```

The first invariant $\mathbf{x} = (f^{\text{itr}}) \mathbf{x0}$ states that $\mathbf{x}$ maintains the trajectory of x_0 under f , that is $x = f^n(x_0)$, and $\mathbf{x_new} = f \mathbf{x}$ declares that $\mathbf{x_new}$ is always the next element, that is $\mathbf{x_new} = f^{n+1}(x_0)$. The material implication $(\text{Break} \neq 0 \longrightarrow |\mathbf{x_new} - \mathbf{x}| < \text{tol})$ essentially flips the line **if** $|\mathbf{x_new} - \mathbf{x}| < \text{tol}$ **then** $\text{Break} := 1$ from the program. Indeed, either $\text{Break} = 0$ or $|\mathbf{x_new} - \mathbf{x}| < \text{tol}$. Finally, clearly itr

$< \text{max_iter}$ except when they are possibly equal which violates the **while**-guard and the program terminates.

Having established that the FPM converges in c -contractive neighbourhoods, we analyse when such neighbourhoods exist. If one can show that $g \in C^1(U)$ for some open set U containing the fixed point r such that $|g'(r)| < 1$ then there is a δ -neighbourhood of r where $|g'(x)| < 1 - \varepsilon$ and g is $(1 - \varepsilon)$ -contractive. Informally, as g' is continuous there exists a δ -neighbourhood S of r where for every $x \in S$, $|g'(x)| < 1 - \varepsilon$ for some $\varepsilon > 0$. Then for all $x, y \in [r - \delta, r + \delta]$, by the mean value theorem there exists $\theta \in [x, y]$ such that:

$$\begin{aligned} |g(x) - g(y)| &= |g'(\theta)| |x - y| \\ &< (1 - \varepsilon) |x - y|. \end{aligned}$$

The formal statement of this result is:



```

1 lemma fixed_point_iter_error_bound_C1:
2   fixes f :: "real ⇒ real" and r tol :: real and max_iter :: nat
3   assumes r_fixed : "f r = r"
4     and r_in_U : "r ∈ U"
5     and U_open : "open U"
6     and C1_on_U : "C_k_on 1 f U"
7     and deriv_strict: "|deriv f r| < 1"
8   shows
9     "∃δ>0. ∃ε>0.
10      (∀x0::real. |x0 - r| ≤ δ →
11        H[True] fixed_point_iter (f, x0, tol, max_iter)
12        [|x - r| ≤ (1 - ε) ^ itr * |x0 - r| ∧ itr ≤ max_iter])"
```

Listing 11: Convergence when $g \in C^1(U)$ and $|g'(r)| < 1$

Finally, there is the special case to consider when $g \in C^1(U)$ on the open set U containing r , $g'(r) = 0$, and g is twice differentiable at r .

Informally, by Taylor's theorem with Peano remainder with $n = 2$, when expanding the Taylor series at r we obtain a quadratic approximation

$$f(x) = f(r) + \frac{f(r)''}{2}(x - r)^2 + h_2(x)(x - r)^2$$

with $h_2(x)$ converging towards 0 as x tends to r .

Assume in addition that

$$r = f(r), \quad x_k = f(x_{k-1}), \quad E_k := |x_k - r|.$$

Now from the previous result, since $|g'(r)| = 0 < 1$, there is a $(1 - \varepsilon)$ -contractive neighbourhood of radius δ for some δ and ε . Observe that:

$$E_k = |x_k - r| = |f(x_{k-1}) - f(r)| = \left| \frac{f''(r)}{2} + h_2(x_{k-1}) \right| (x_{k-1} - r)^2.$$

Since $\lim_{x \rightarrow a} h_2(x) = 0$, there exists $d > 0$ such that

$$|x - r| < d \implies |h_2(x)| < \frac{\varepsilon}{2}.$$

To inherit both the contractive property and the limit property, we take the minimum of both quantities:

$$a = \min(d, \delta),$$

so that when $|x - r| < a$, x is in a contractive neighbourhood and $|h_2(x)| < \varepsilon/2$. Hence if $|x_{k-1} - r| < a$, we get

$$\begin{aligned} E_k &= \left| \frac{f''(r)}{2} + h_2(x_{k-1}) \right| (x_{k-1} - r)^2 \\ &< \left(\frac{|f''(r)|}{2} + \frac{\varepsilon}{2} \right) (x_{k-1} - r)^2 =: C E_{k-1}^2. \end{aligned}$$

By induction one shows

$$\begin{aligned} E_k &\leq C E_{k-1}^2 \\ &\leq C (C E_{k-2}^2)^2 \\ &= C^{1+2} E_{k-2}^{2^2} \\ &\leq \dots \\ &\leq C^{1+2+\dots+2^{k-1}} E_0^{2^k}. \end{aligned}$$

Using the geometric-series sum $1 + 2 + \dots + 2^{k-1} = 2^k - 1$, this simplifies to the quadratic-convergence estimate

$$E_k \leq \left(\frac{|f''(r)| + \varepsilon}{2} \right)^{2^k - 1} E_0^{2^k}.$$

We state this result formally:

```
1 lemma fixed_iter_quadratic_convergence_case :
2   fixes f      :: "real ⇒ real"
3   and r        :: real
4   and max_iter :: nat
```

```

5  assumes r_fixed      : "f r = r"
6      and r_in_U       : "r ∈ U"
7      and cont_deriv    : "C_k_on 1 f U"
8      and der0         : "deriv f r = 0"
9      and twice_dff     : "f twice_differentiable_at r"
10 shows
11   "∃(δ :: real)>0. ∃(ε :: real)>0.
12     (∀x0. |r - x0| < δ →
13       H[True] fixed_point_iter (f,x0, tol, max_iter)
14       [|x - r| ≤ (((|deriv (deriv f) r| + ε)/2)^(2^itr-1))* |x0 -
15         r|^(2^itr) ∧ (itr ≤ max_iter)])]"

```

Listing 12: Quadratic convergence when $g \in C^1(U)$, g twice differentiable at r , and $g'(r) = 0$

Due to our work in previous sections, the formalisation and verification of the fixed point method (and the results described in this section) span approximately 1080 lines of Isabelle code without comments or spaces.

8 Evaluation

Having discussed the verification of the bisection and the fixed point method, we proceed to evaluate Isabelle as the foundation for that task. Our evaluation is framed around 3 research questions:

1. How suited is Isabelle to modelling numerical methods?
2. How easy is to iterate on the verification and to specify the invariants for it?
3. How automated is the verification process?

With the bisection method, Isabelle executes the program smoothly; and, on simple polynomials of degree three, the output of applying **execute** to **bisection** were equivalent to results of bisection performed in R and Python. The bisection program required 9 invariants, naturally occurring in 6 parts. The first two “parts” were invariants which essentially appeared in the code itself. The final invariant was the most unnatural because it required the disjunction $\text{itr} = 0 \vee 2(\text{upper} - \text{lower}) > \text{tol}$. The latter disjunct required multiplying by 2 as a way to reason about what **upper** and **lower** were in the previous iteration, a particularly unnatural and challenging line of reasoning. After establishing this invariant, it soon became apparent that if $\text{itr}=0$ there was no previous iteration, thus it was added to the disjunction.

When using `vcg` to prove `bisection_error_bound`, it generated 22 goals, 20 ($\approx 91\%$) of which were automatically proven by `Sledgehammer`. The last two goals were satisfied with user-supplied Isar proofs of about 40 lines, and these goals were the same but different “cases”. Lastly, these two final goals required `Bolzanos_theorem`, a special case of `IVT'` (the Intermediate Value Theorem) where the intermediate value is 0.

The FPM program required 4 invariants all of which were did not require us much time to state. There were 3 main theorems we proved about the FPM. For `fixed_point_known_iter_error_bound_local`, there were 3 goals, 2 of which were essentially the same up to case analysis, and all 3 required relatively short Isar proofs. After introducing the higher-order differentiation language, `fixed_point_iter_error_bound_C1` was demonstrated by a straight forward Isar proof which required showing we could generate the hypotheses needed for the previous lemma. Finally, we use our Taylor Theorem with Peano remainder before applying `vcg` to prove the statement `fixed_point_iter_quadratic_convergence_case`. The `vcg` method generates two goals corresponding to the informal inductive proof above.

So to answer our research questions, (1) Isabelle has indeed been a useful tool for the modelling and analysis of the numerical methods presented here. (2) The Isabelle/ITrees framework has been useful for quickly iterating our verification tasks and refining its invariants, and (3) the presence of `Sledgehammer` and the `vcg` method enable a high degree of automation.

9 Related Work

Verification of numerical methods has been an ongoing area of interest for the formal methods community. In PVS, for instance, there has been work towards building libraries for reasoning about real numbers since at least the mid-2000s [6, 21, 20, 9]. However, those works have focused on interval arithmetic and Taylor models as abstractions for reasoning about polynomial and real arithmetic inequalities. In contrast, we leverage the recent development of analysis libraries and directly reason about Taylor polynomials, rather than models. In the Rocq proof assistant (formerly Coq), early work towards verification of numerical methods analysed Kantorovitch’s theorem and Newton’s method using similar methods of interval arithmetic, Taylor models and linear algebra [23]. Its verification of Kantorovitch’s theorem required the formalisation of Taylor’s theorem up to the second derivative. As seen above, our work uses arbitrary higher-order differentiation.

More recently, in Rocq, a framework is being developed for verifying

numerical methods and producing reliable C programs [17, 1]. Just like our work, the project leverages the libraries of a long-standing prover. For instance, it relies on the **Taylor Lagrange** theorem from the Coquelicot library [2]. Yet, its verification examples have focused on the harmonic oscillator, while we focus here on the bisection and the fixed-point iteration methods. An alternative, younger method using Taylor models and defining higher-order differentiation focuses on approximating solutions to initial value problems for non-linear polynomial ordinary differential equations (ODEs) [22]. It arbitrarily approximates the solution to the ODE by using more terms of its Taylor model.

In Isabelle/HOL itself, there have been verifications for specific algorithms, but not a framework per se. For instance, some Runge-Kutta methods over deep embeddings of arithmetic expressions or set-based Euler methods have been verified [12, 13]. We have also compared extensively the higher-order differentiability of the Smooth Manifolds AFP entry [16] with our definitions in Section 5. Furthermore, as the name suggests, the focus there is on formalising smooth manifolds instead of verifying numerical algorithms.

Finally, in Lean, the rapid growth of the MathLib library [19] has caught up with older provers like Rocq and Isabelle in terms of formalisation of higher-order derivatives, Taylor series and continuously differentiable functions. Yet, as far as we know, a verification framework for numerical algorithms is still nonexistent there.

10 Conclusion

We have presented a framework for verification of numerical methods, based on the Isabelle/ITrees library, and have applied it to two important methods: Bisection and the Fixed-Point Method. The **vcg** method is well-suited for automating the verification process of program specifications. Notably, one does not need to understand how **vcg** works or even be familiar with Hoare logic beyond stating specifications with Hoare triples, to prove otherwise very complicated goals, provided the correct loop invariants are stated.

With the correct definitions for k -differentiability and $C^k(U)$ in place, we hope to fully characterize the smoothness of the sigmoid function and prove the Universal Approximation Theorem as part of our future work. Moreover, there are many opportunities to extend the differentiation library, including a full treatment of the Hessian and weak (distributional) derivatives. We believe that our work can become the foundation for a rigorous

collaborative classification of the various derivative definitions in the libraries of formalised mathematics and of mathematics in general.

We aim to build on previous AFP entries [3] to advance the formal verification of optimisation theory in Isabelle. The existing development is largely focused on the unconstrained, one-dimensional case. A robust multi-dimensional, unconstrained and constrained optimisation theory would be a much welcome aide to the downstream verification of artificial intelligence (AI) and machine learning algorithms, as well as an avenue for explainable AI. Finally, we believe that the work demonstrated in this essay could motivate others to model historically significant structures, like the perceptron, and verify the correctness of their training algorithms. Specifically, contemporary machine learning algorithms and their properties, such as stochastic gradient descent and ADAM, are tractably analyzable in Isabelle.

Funding statements: A Horizon MSCA 2022 Postdoctoral Fellowship (project acronym DeepIsaHOL and number 101102608) supported the second author during the development of this article.

References

- [1] Andrew W. Appel and Ariel Kellison. VCFLOAT2: Floating-point error analysis in Coq. In Amin Timany, Dmitriy Traytel, Brigitte Pientka, and Sandrine Blazy, editors, *CPP 2024*, pages 14–29. ACM, 2024. <https://doi.org/10.1145/3636501.3636953>.
- [2] Sylvie Boldo, Catherine Lelay, and Guillaume Melquiond. Coquelicot: A user-friendly library of real analysis for coq. *Mathematics in Computer Science*, 9:41–62, 2015. <https://doi.org/10.1007/s11786-014-0181-1>.
- [3] Dustin Bryant. Unconstrained optimization. *Archive of Formal Proofs*, July 2025. https://isa-afp.org/entries/Unconstrained_Optimization.html, Formal proof development.
- [4] Dustin Bryant, Jonathan Julián Huerta y Munive, and Simon Foster. Numerical methods in Isabelle, November 2025. <https://doi.org/10.5281/zenodo.17679526>.
- [5] Dustin Bryant, Jim Woodcock, and Simon Foster. The sigmoid function and the universal approximation theorem. *Archive of For-*

- mal Proofs*, May 2025. https://isa-afp.org/entries/Sigmoid_Universal_Approximation.html, Formal proof development.
- [6] Marc Daumas, David Lester, and César Muñoz. Verified real number calculations: A library for interval arithmetic. *IEEE Transactions on Computers*, 58(2):226–237, February 2009.
 - [7] S. Foster, J. Baxter, A. Cavalcanti, J. Woodcock, and F. Zeyda. Unifying semantic foundations for automated verification tools in Isabelle/UTP. *Science of Computer Programming*, 197, October 2020.
 - [8] S. Foster, C.-K. Hur, and J. Woodcock. Unifying model execution and deductive verification with Interaction Trees in Isabelle/HOL. *ACM Trans. on Software Engineering Methodology (TOSEM)*, 2024.
 - [9] Hanne Gottliebsen, Ruth Hardy, Olga Lightfoot, and Ursula Martin. Applications of real number theorem proving in PVS. *Formal Aspects Comput.*, 25(6):993–1016, 2013. <https://doi.org/10.1007/s00165-012-0232-9>.
 - [10] Johannes Hölzl, Fabian Immler, and Brian Huffman. Type classes and filters for mathematical analysis in Isabelle/HOL. In *ITP*, volume 7998 of *LNCS*, pages 279–294, Heidelberg, 2013. Springer.
 - [11] Jonathan Julián Huerta y Munive, Simon Foster, Mario Gleirscher, Georg Struth, Christian Pardillo Laursen, and Thomas Hickman. IsaVODEs: Interactive verification of cyber-physical systems at scale. *Journal of Automated Reasoning*, 68(4):21, 2024.
 - [12] Fabian Immler. Formally verified computation of enclosures of solutions of ordinary differential equations. In *NFM 2014*, volume 8430 of *Lecture Notes in Computer Science*, pages 113–127, Heidelberg, 2014. Springer.
 - [13] Fabian Immler. A verified ODE solver and the lorenz attractor. *J. Autom. Reason.*, 61(1-4):73–111, 2018.
 - [14] Fabian Immler and Johannes Hölzl. Numerical analysis of ordinary differential equations in Isabelle/HOL. In *ITP*, volume 7406 of *LNCS*, pages 377–392, Heidelberg, 2012. Springer.
 - [15] Fabian Immler and Christoph Traut. The flow of ODEs: Formalization of variational equation and poincaré. In *ITP 2016*, volume 9807 of *LNCS*, pages 184–199, Heidelberg, 2016. Springer.

- [16] Fabian Immler and Bohua Zhan. Smooth manifolds and types to sets for linear algebra in Isabelle/HOL. In *CPP 2019*, pages 65–77. ACM, 2019. <https://doi.org/10.1145/3293880.3294093>.
- [17] Ariel E. Kellison and Andrew W. Appel. Verified numerical methods for ordinary differential equations. In *FoMLAS 2022, and NSV 2022*, volume 13466 of *LNCS*, pages 147–163. Springer, 2022. https://doi.org/10.1007/978-3-031-21222-2_9.
- [18] Vitalii Konarovskiy. Lecture notes for the course linear algebra and calculus of functions of one variable. Universitat Leipzig, 2018. https://konarovskiy.de/teaching/2018/Math1/Math1_2018.html.
- [19] The mathlib Community. The lean mathematical library. In *CPP 2020*, CPP 2020, page 367–381, New York, NY, USA, 2020. Association for Computing Machinery.
- [20] César Muñoz and David Lester. Real number calculations and theorem proving. In J. Hurd and T. Melham, editors, *Proceedings of the 18th International Conference on Theorem Proving in Higher Order Logics, TPHOLs 2005*, volume 3603 of *Lecture Notes in Computer Science*, pages 195–210, Oxford, UK, 2005. Springer-Verlag.
- [21] Anthony Narkawicz and César Muñoz. A formally verified generic branching algorithm for global optimization. In Ernie Cohen and Andrey Rybalchenko, editors, *Proceedings of the 5th International Conference on Verified Software: Theories, Tools, and Experiments (VSTTE 2013)*, volume 8164 of *LNCS*, pages 326–343, Menlo Park, CA, US, May 2014. Springer.
- [22] Sewon Park and Holger Thies. A Coq formalization of taylor models and power series for solving ordinary differential equations. In *ITP 2024*, volume 309 of *LIPICs*, pages 30:1–30:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.
- [23] Ioana Pasca. *Formal Verification for Numerical Methods. (Vérification formelle pour les méthodes numériques)*. PhD thesis, University of Nice Sophia Antipolis, France, 2010. <https://tel.archives-ouvertes.fr/tel-00555158>.
- [24] Esteban I. Poffald. The remainder in taylor’s formula. *The American Mathematical Monthly*, 97(3):205–213, March 1990. JSTOR stable URL; accessed 2016-02-09.

- [25] Walter Rudin. *Principles of Mathematical Analysis*, pages 110–111. McGraw-Hill Book Co., Singapore, 3 edition, 1976.
- [26] Justin Solomon. *Numerical Algorithms: Methods for Computer Vision, Machine Learning, and Graphics*. A K Peters/CRC Press, Boca Raton, FL, 1 edition, July 2015. https://people.csail.mit.edu/jsolomon/share/book/numerical_book.pdf.
- [27] Michael Spivak. *Calculus*. Cambridge University Press, Cambridge, 3rd edition, 1994.